

Securing IoT: Unveiling Attacks With Multiview-Multitask Learning

Urkhimbam Bobby Clinton , *Member, IEEE*, Nazrul Hoque , Shahid Raza , *Senior Member, IEEE*, and Monowar Bhuyan , *Senior Member, IEEE*

Abstract—With the rapid expansion and use of day-to-day Internet of Things (IoT) applications, cybercriminals are exploiting an increasingly wide attack surface and are capable of executing many successful attacks, leading to significant losses for individuals and organizations. The existing defense mechanisms often rely on single-view, single-task models that utilize a single feature set to perform one specific task. However, modern IoT systems are multifaceted, heterogeneous, and resource-constrained, posing considerable challenges for developing unified and scalable defense solutions. To address this, we propose a novel hybrid model called M²VT that integrates multiview learning (MVL) and multitask learning (MTL) for effective cyber-attack defense. The model simultaneously processes three distinct subsets of relevant features (views) to perform three interrelated tasks: attack detection, attack category classification, and attack type classification. The model leverages autoencoder (AE) and long short-term memory (LSTM) networks to extract task-specific spatial and temporal features, enhancing both efficiency and cost-effectiveness. The M²VT model is evaluated using four publicly available IoT benchmark datasets and one testbed dataset. Across all tasks and datasets, the model achieves over 96% accuracy, consistently outperforming state-of-the-art approaches. The parallel execution of MVL and MTL, combined with task-specific feature subsets, significantly boosts performance. The implementation of M²VT is publicly available in our code repository.¹

Impact Statement—The proposed M²VT model has significant impacts in unveiling IoT attacks, categories, and types through parallel execution of MVL and MTL. This parallel learning strategy makes M²VT both accurate and cost-effective. The approach provides a robust and impactful defense against IoT vulnerabilities, achieving up to 100% detection accuracy and a detection speed three times faster than existing methods, making it suitable for real-time deployment.

Index Terms—Autoencoder, Internet of Things (IoT), long short-term memory (LSTM), multitask learning (MTL), multiview learning (MVL), network traffic.

I. INTRODUCTION

THE Internet of Things (IoT) has experienced rapid growth due to its wide range of applications and daily uses that impact nearly every aspect of day-to-day life. However, the increasing number of connected devices in IoT networks, coupled with their inherent vulnerabilities, has made them attractive targets for cybercriminals. These adversaries exploit IoT systems for various malicious purposes, including botnet formation, data theft, ransomware attacks, spamming, and cryptojacking [1]. Such cyber-attacks can lead to severe consequences, including financial losses, data breaches, service disruptions, and violations of privacy for both individuals and organizations [2]. The heterogeneity of devices and networks, along with resource constraints, poses significant challenges for conventional defense systems in addressing emerging IoT-specific threats [3], [4]. This necessitates the development of lightweight and adaptive security solutions capable of protecting IoT infrastructures. To this end, researchers have proposed intrusion detection systems (IDS), intrusion prevention systems (IPS), and intrusion mitigation systems (IMS), utilizing statistical methods, machine learning (ML), and deep learning (DL) techniques to address them.

While ML and DL models have shown promise, their effectiveness heavily depends on the quality and representation of the data used for training. Traditional ML approaches require manual feature engineering, whereas DL models can automatically learn hierarchical representations from raw data. To improve model performance and efficiency, various techniques such as feature extraction, feature selection, and dimensionality reduction are employed to derive informative and relevant input features. Although classical ML methods generally incur lower computational costs, DL approaches are often preferred due to

Received 14 September 2024; revised 26 January 2025 and 12 June 2025; accepted 22 September 2025. Date of publication 1 October 2025; date of current version 31 March 2026. This work was supported in part by the Start-up-Grant of DST-SERB bearing File no.: SRG/2022/001692, and in part by Wallenberg AI, Autonomous Systems and Software Program (WASP) by Knut and Alice Wallenberg Foundation and the European Commission through the Horizon Europe project COGNIT under Grant 101092711. This article was recommended for publication by Associate Editor Chaker Abdelaziz Kerrache upon evaluation of the reviewers' comments. (*Corresponding author: Nazrul Hoque.*)

Urkhimbam Bobby Clinton and Nazrul Hoque are with the Department of Computer Science, Manipur University (a Central University), Imphal 7950032, India (e-mail: bobby.phd.csc@manipuruniv.ac.in; tonazrul@gmail.com).

Shahid Raza is with the School of Computing Science, University of Glasgow, G12 8RZ3 Scotland, U.K., and also with RISE, 16440 Stockholm, Sweden (e-mail: shahid.raza@glasgow.ac.uk).

Monowar Bhuyan is with the Department of Computing Science, Umeå University, SE-90187 Umeå, Sweden (e-mail: monowar@cs.umu.se).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2025.3615565>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2025.3615565

¹<https://github.com/3Clinton/MVMTL>

their superior ability to handle large-scale data and automatically extract both spatial and temporal patterns. In particular, deep learning has demonstrated strong performance in applications like network traffic classification, anomaly detection, and network optimization [5]. Among DL architectures, autoencoders (AEs) and long short-term memory (LSTM) networks have proven effective for processing IoT data. Autoencoders are neural networks designed to compress input data into a lower-dimensional representation and reconstruct it, preserving essential information. They are widely used for dimensionality reduction, anomaly detection, and data denoising. Compared to traditional methods such as principal component analysis (PCA), AEs are more flexible and capable of capturing nonlinear relationships in data, albeit at a higher computational cost. LSTM networks, a variant of recurrent neural networks (RNNs), are specifically designed to handle sequential data and extract temporal dependencies. They effectively address long-term dependencies in time-series data and mitigate the vanishing gradient problem, making them highly suitable for analyzing dynamic IoT network behavior.

Although conventional single-view, single-task deep learning (DL) models are inspired by the human learning process, they fall short of truly replicating it. Biologically, humans can accurately recognize or classify objects by observing them from multiple perspectives or based on diverse perceptions. Furthermore, humans often perform tasks more efficiently by leveraging knowledge gained from related or similar tasks. To approximate this level of cognitive capability, multiview learning (MVL) and multitask learning (MTL) have emerged as effective paradigms. MVL enables models to learn from multiple views or feature sets, while MTL allows for the simultaneous learning of multiple related tasks. These approaches not only enhance model performance but also reduce computational overhead compared to conventional DL models. The concepts of MVL and MTL, as well as their integration in our approach to uncover IoT attacks through behavioral network traffic analysis, are detailed in Sections III-A and III-B.

In this article, we propose a novel lightweight and hybrid attack detection system for IoT networks that leverages multiview learning (MVL) and multitask learning (MTL). The system utilizes task-specific subsets of relevant features as distinct views, allowing the construction of a more generalized model that benefits from shared information across multiple tasks and concurrently performs multiple tasks: attack detection, attack category classification, and attack types classification. To enhance performance, the system incorporates an autoencoder (AE) to extract spatial features and reduce input dimensionality, as well as a long short-term memory (LSTM) network for capturing temporal dependencies in sequential data.

Over the past decades, numerous intelligent attack detection and mitigation systems have been developed for IoT networks, utilizing statistical methods, machine learning (ML), and deep learning (DL) techniques. However, most existing systems rely on a single set of features to perform a specific task—either attack detection or classification. Due to the heterogeneous nature of IoT network traffic, selecting a unified feature set for such tasks remains a significant challenge. Moreover, designing

a lightweight system that can detect and classify attacks concurrently while maintaining high performance is still an open issue. Although some existing approaches incorporate multiview learning (MVL) or multitask learning (MTL) individually, to the best of our knowledge, no prior work has integrated both MVL and MTL for securing IoT networks against cyber-attacks. The proven effectiveness of autoencoders (AEs), long short-term memory (LSTM) networks, MVL, and MTL in various application domains inspired us to develop a hybrid smart attack detection system that combines all these techniques. By leveraging MVL and MTL, the system can utilize multiple feature sets to perform multiple tasks concurrently, enhancing the model's generalization and improving performance in detecting and classifying cyber-attacks. Furthermore, the integration of AE and LSTM enables the extraction of meaningful spatial representations and the capture of temporal dependencies, further boosting the model's effectiveness.

The key contributions of this work are as follows.

- 1) To the best of our knowledge, this is the first study to leverage both multiview learning (MVL) and multitask learning (MTL) to detect cyber-attacks by analyzing network-wide IoT traffic behaviors.
- 2) We propose a hybrid model called M²VT that utilizes multiple feature sets (views) to perform multiple tasks concurrently: attack detection, attack category classification, and attack type classification in IoT network traffic.
- 3) The model employs a wrapper-based feature selection method to identify task-relevant features, which are treated as separate views. Autoencoders (AEs) are used on each view to extract spatial features and reduce dimensionality. A shared long short-term memory (LSTM) layer then captures temporal dependencies across tasks. By combining the output of each AE with the shared LSTM, the model performs all tasks concurrently and efficiently.
- 4) The proposed M²VT model is evaluated using four publicly available benchmark IoT datasets and a real-time testbed dataset, and compared with state-of-the-art approaches. The M²VT model outperforms all baseline models, achieving up to 1.45% higher accuracy than the best-performing state-of-the-art method.

II. RELATED WORK

Developing network intrusion detection and prevention systems for securing IoT environments from cyber-attacks has been a topic of significant interest in recent years. Many detection and prevention of emerging cyber-attacks based on statistical, ML, DL, and hybrid approaches are found in the literature. Moreover, DL-based multiview learning methods have gained popularity in handling various problems such as intrusion detection, text classification, 3-D reconstruction, facial detection, and human activity recognition [6]. Similarly, multitask learning methods are also found to be very effective in the applications of computer vision, bioinformatics, health informatics, speech, NLP, web, and so on [7], [8]. Next, we discuss both multiview

TABLE I
COMPARISON AMONG EXISTING WORKS

Approach	Ref.	View		Task		Tested on		Dataset Type		Dataset Used		Classification Type		Performed Feature Selection	Handled Data Imbalance
		Single	Multi	Single	Multi	CPU	GPU	IoT	IIoT	Benchmark	Own Dataset	Binary	Multiclass		
ML	[21]	✓		✓			✓	✓	✓	✓		✓			
	[22]	✓		✓		✓		✓		✓		✓		✓	
	[23]	✓		✓		✓	✓	✓		✓		✓	✓	✓	
	[24]	✓		✓			✓	✓	✓	✓		✓		✓	
DL	[25]	✓		✓		✓		✓		✓		✓	✓		✓
	[26]	✓		✓		✓		✓		✓			✓	✓	✓
	[27]	✓		✓		✓		✓		✓			✓		
	[28]	✓		✓		✓		✓	✓	✓		✓	✓		
	[29]	✓		✓		✓				✓	✓	✓			
MVL	[9]		✓	✓		✓				✓		✓	✓		
	[10]		✓	✓			✓	✓		✓		✓		✓	
	[11]		✓	✓		✓	✓			✓		✓			
MTL	[14]	✓			✓		✓	✓			✓	✓	✓	✓	✓
	[16]	✓			✓	✓				✓		✓			✓
MVL+MTL	[30]	✓	✓	✓	✓	✓		✓		✓		✓	✓		
	proposed	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

and multitask learning approaches used for IoT network traffic analysis.

A. MVL-Based Approaches

Rather than a long feature vector used in the conventional DL approaches, He et al. [9] consider three different levels of features from network data, such as basic (attributes based on protocol connection), content (attributes of the payload information), and traffic (attributes based on connection time). The authors incorporated these features to develop an MVL model called MS-DHPN, which can detect intrusions in a conventional network. MS-DHPN consists of two layers: a multimodal DAE (MDAE) to integrate the complex features in each traffic flow at the low level and an LSTM layer to extract the temporal information between traffic flows at the high level. Their experimental results show the dominance of the MVL approach over the single-view approach on the NSL-KDD, UNSW-NB15, and CICIDS-2017 datasets. Training a model with a single view may result in perceiving incomplete knowledge of patterns in large features of a dataset. Moreover, the conventional model's training approaches work in a centralized manner that receives data from multiple edge nodes. This centralized storage might result in serious breaches in data privacy and security.

Attota et al. [10] developed an IDS based on federated learning (FL) with ensemble MVL called MV-FLID. The model used multiple views of IoT network data during the model's training in a decentralized approach. They employ the ensemble MVL and FL to maximize the learning efficiency of different attack types and to preserve data privacy. The experiment is carried out on the MQTTset dataset, extracting three views, i.e., packet view, unifiow view, and biflow view, using the Integrating grey wolves optimization mechanism. Their result shows the effectiveness of FL with MVL over the single view and centralized training approach. In addition, MVL has been used to detect malware attacks [11], phishing websites on the internet [12], and attacks in industrial control systems [13]. Although the

MVL has been used in many application domains, it is still early in the domain of IoT cybersecurity.

B. MTL-Based Approaches

Sajid Ali et al. [14] propose an MTL model for detecting malware in an IoT environment. The model incorporates an LSTM to perform malware detection as task 1 and identification of malware types as task 2. They performed a time-series analysis on the packets of network traffic flows containing both normal and malicious traffic collected from 18 IoT devices. Moreover, they extracted features from the raw network traffic data and grouped them into three categories, i.e., packet payload-related, flow-related, and traffic flag-related features. Further, they applied FS to select the relevant features from each group and merge them to use during the training of their model, improving the model's performance. Similarly, Laisen Nie et al. [15] developed an MTL method using LSTM to predict the IIoT backbone network traffic accurately. Using MTL with an over-sampling technique, Lijian Sun et al. [16] developed a solution to handle the data imbalance and achieve high performance in IoT networks' high-variation data sources. Their experiment is performed using the UNSW-NB15 and CICIDS-2018 datasets and shows the effectiveness of their model over the single-task learning models. Moreover, to detect network intrusions effectively, Qigang et al. [17] also developed an MTL approach that comprises a supervised learning-based clustering model, an AE-based contrastive learning model, and an MLP-based classifier. They extend their work by exploring important features for each particular task and also develop a loss function to handle the data imbalance. Their approaches show the effectiveness of MTL in network intrusion detection. Further, Huang et al. [18], [19], and Li et al. [20] also developed MTL methods to detect intrusions and malware effectively. However, these approaches are still computationally expensive for IoT devices.

Although various methods or approaches exist, as shown in Table I, we have not found any work that leverages both MVL and MTL to build a smart attack detection and mitigation

system to secure the IoT network. Moreover, many authors used non-IoT datasets to experiment with their methods or approaches, which leads to low performance and a high false alarm rate when deployed into real IoT environments. So, in this article, we present a hybrid model that leverages both MVL and MTL to secure the IoT network.

C. Problem Statement

Let's consider the variables F , C , V , and T to represent features, class labels, views, and tasks, respectively. For the set of views $V = \{v_1, v_2, v_3\}$, corresponding to the network traffic features, $F = \{f_1, f_2, \dots, f_n\}$, and class labels $C = \{c_1, c_2, c_3\}$, we develop a multiview-multitask model M , such that M computes every task, $t_i \in T$, where t_i is to perform the prediction of $c_i \in C$, with respect to the required views, $v_i \in V$. Hence, the model M is defined as $MV(F) \rightarrow T$.

III. M²VT APPROACH

We proposed a hybrid smart attack detection system to detect and classify attacks from cyber-criminals in IoT networks by utilizing MVL and MTL. The system takes the feature subsets that are highly relevant to the respective class labels as multiple views. The system learns the shared information from these multiple views to perform the respective tasks concurrently. The proposed model aims to perform three related tasks, i.e., Task 1: detecting attacks (binary classification), Task 2: classification of attack categories (multiclass classification), and Task 3: classification of attack types (also a multiclass classification). All three tasks are related to each other in terms of attack behavior and attack dimension. Moreover, the system incorporates AE to extract spatial information as well as to reduce the dimension of the input views and LSTM to extract the temporal information between the related tasks.

Definition 3.1: View: A view $v_i \in V$ can be defined as a set of optimal features used to perform a task $t_i \in T$ using a model M . The model considers different views to perform different tasks. The views are generated from the spatial domain and the temporal domain.

Definition 3.2: Task: A task $t_i \in T$ is an operation performed by a model M with a set of features $f_i \in F$ on a Dataset D . The operation may be a classification or a prediction. In our approach, tasks are one binary classification and two multiclass classifications.

Definition 3.3: Multitask Learning (MTL): For a given set of tasks $T = \{t_1, t_2, \dots, t_n\}$, a learning model M performs the tasks simultaneously, considering a set of n losses such that the learning operation is performed with some optimization to the losses.

There are four possible combinations of learning approaches in terms of multiview and multitask learning. For a set of views $V = \{v_1, v_2, \dots, v_n\}$ and a set of tasks $T = \{t_1, t_2, \dots, t_n\}$, we can define the four learning approaches using a learning model, say M as follows.

1) *Single-view single-task*: Here, the learning model M takes a view (v_i), which is a set of features, and performs

only one task (t_i), which is a prediction. Mathematically, we can represent it as $M(v_i) \Rightarrow t_i$.

2) *Single-view multiple-tasks*: Here, the learning model M takes a view (v_i), which is a set of features, and performs multiple tasks ($t_i \in T$), which are predictions. Mathematically, we can represent it as $M(v_i) \Rightarrow T$.

3) *Multiple-views single-task*: Here, the learning model M takes multiple views ($v_i \in V$), which are multiple sets of features, and performs only one task (t_i), which is a prediction. Mathematically, we can represent it as $M(V) \Rightarrow t_i$.

4) *Multiple-views multiple-tasks*: Here, the learning model M takes multiple views ($v_i \in V$), which are multiple sets of features, and performs multiple tasks ($t_i \in T$), which are predictions. Mathematically, we can represent it as $M(V) \Rightarrow T$.

A. Multiview Learning

MVL is an emerging paradigm in ML or DL that mimics the human learning process in real-world scenarios where different perspectives or views of the data can provide supporting information during the learning process [6]. MVL leverages information from multiple views to improve the model's generalization performance. In MVL, each view represents a different perspective of data and assumes that the combination of these views can lead to a more inclusive and accurate understanding of the underlying patterns, which is required when data is heterogeneous and different views provide diverse information.

In this work, we consider different subsets of features from the available features as different views, i.e., a feature subset for performing attack detection as one view, attack types as another view, and so on. We select the respective feature subsets as views based on their relevance to the particular tasks. For example, the features that are highly relevant in detecting an attack are considered as a view. To identify the highly relevant features, we leverage the wrapper-based FS method (forward selection) using the random forest as an estimator. This FS method selects the optimal feature subsets for each respective class label. The purpose of incorporating feature selection is to address the fact that, even when tasks or class labels are related, a subset of features that is highly relevant to one particular task may not necessarily be relevant to another related task. Since all tasks are interconnected, comprehensive information can be derived by combining the relevant subset of features for a specific task with the subsets relevant to the remaining related tasks. This combined information encompasses all necessary details for each task, making multitask learning an effective approach. We chose random forest (RF) as the estimator due to its ability to handle nonlinear relationships, its inherent feature importance ranking, and its robustness in dealing with high-dimensional data. Random Forest effectively mitigates challenges such as the risk of overfitting and computational complexity, making it a suitable choice for feature selection in our study. Moreover, we incorporate respective AE into each view to extract spatial features and reduce the dimensions of

the views. Further, these views are processed using MTL, as explained below.

In multiview learning, the AE extracts the set of spatial features from the given input views. The spatial features are merged together in the concatenation layer using an append operation. The append operation combines all the values given as output by the ReLu activation function used in the latent layer of the AE. The values obtained from the multiple views having redundant features will not yield redundant information at the concatenation layer, as ReLu's output for the same feature presence in different views yields different information.

Mathematically, we discuss the functionality of the concatenation as follows. For a view $v_i \in V$, the output achieved from the AE is a lower-dimensional spatial feature vector (sp_i) containing spatial information of v_i . Let Append() be an append function that takes all the outputs, i.e., the set of spatial feature vectors $sp_i \forall v_i \in V$, as inputs, and generates the combined feature vector

$$C_{\text{vectors}} = \text{Append}(sp_i). \quad (1)$$

The C_{vectors} keeps the actual spatial information of the multiple views irrespective of the actual input feature space.

B. Multitask Learning

MTL learns shared information to perform multiple tasks adaptively that provide improved model generalization, which was initially introduced by Rich Caruana [31]. Moreover, it is also used in many domains of IoT, such as anomaly detection, network traffic analysis, and cross-domain learning. In DL, MTL can be performed either by hard or soft parameter sharing of hidden layers. However, a common approach is to use a shared feature extractor and multiple heads for each specific task. The shared feature extractor is used to extract features from the input data and is shared across all the tasks. The heads for each specific task are to perform respective tasks (predictions) and are typically connected to the shared feature extractor. In our proposed model, we employed LSTM layers as the shared feature extractor to extract the temporal information and three task-specific heads to perform Task 1: attack detection, Task 2: classification of attack categories, and Task 3: classification of attack types.

In our multitask learning, the shared layers extract the temporal features from the combined spatial information using the LSTM network with the ReLu activation function. Next, the learning operation for multiple tasks is performed at the concatenation layers of the task-specific heads. The concatenation layer takes two inputs, viz., temporal features and spatial features. Now, similar to the concatenation layer defined in the MVL block, the append operation of the concatenation layers in the MTL block is performed using (1). Unlike other MTL models, we don't define an aggregate optimization technique in our model. Instead, we use the Adam optimization technique as defined in (2) to optimize the incurred loss given by the model on each task

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t. \quad (2)$$

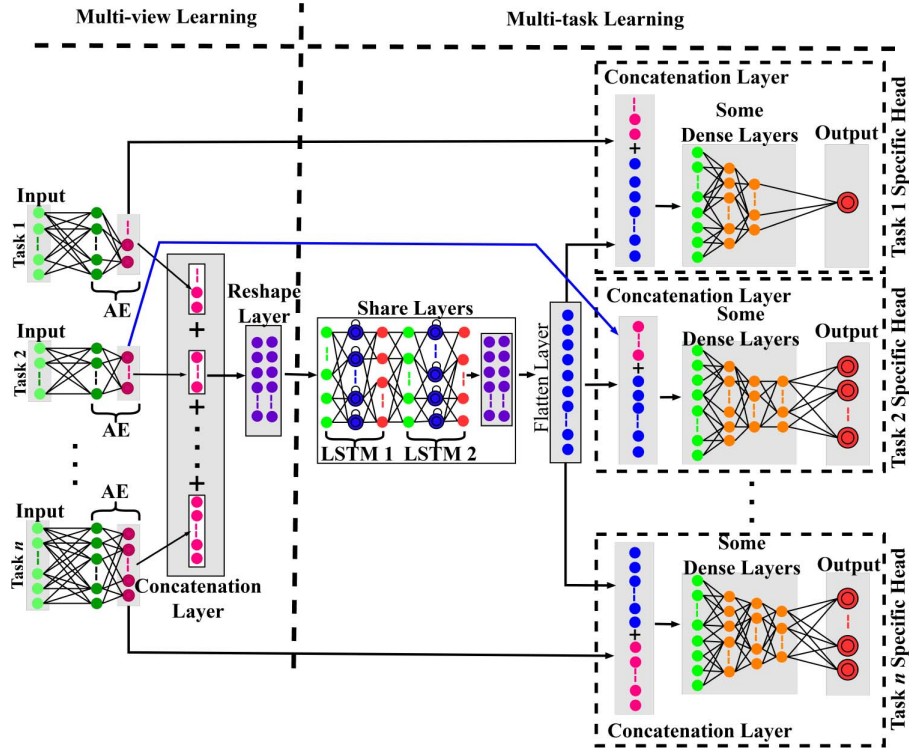
In the above equation, optimized weights at a time slot t are represented as θ_t with a learning rate η , considering a small constant defined as ϵ . This constant plays a major role in keeping the stability of the denominator. Moreover, the other two parameters, viz., $\hat{v}_t$ and $\hat{m}_t$, represent the first and second momentum of the correct bias, respectively.

C. M²VT Model

The proposed hybrid model is known as M²VT, which combines MVL and MTL to obtain a generalized model. The structure of the proposed model is illustrated in Fig. 1. The model is broadly divided into MVL and MTL blocks. In the MVL block, there are three input layers, each for respective views or tasks, followed by three respective AEs. These AEs extract spatial information as well as reduce the dimension of each input. The number of nodes or neurons of the AE layers varies and is adapted accordingly based on the dataset and dimension of the input view. Moreover, AE layers used the "activity_regularizer" parameter with the value "regularizers.l1(10e-5)" and the activation function "ReLU." Later, these outputs of the AEs are grouped together using the "Concatenation Layer." Further, the output from this "Concatenation Layer" is reshaped into the proper dimension using the "Reshape Layer" to fit into the following "Shared Layers" of the MTL block.

In the MTL block, the subblock "Shared Layers" takes the output of the MVL's "Reshape Layer" to extract the temporal information of the input data. This subblock contains two consecutive "LSTM" layers, i.e., LSTM 1 and LSTM 2. These LSTM layers contain 16 units with activation function "ReLU" and "return_sequence" parameters as "True." The "Flatten layer" takes the output of the "LSTM 2" and converts the dimension to fit into the "task-specific heads" subblocks. There are three task-specific subblocks to perform: Task 1, Task 2, and Task 3. The task-specific heads contain a "Concatenation layer," some consecutive "Dense layer" of activation function "ReLU," and an output layer of the particular task. The "Concatenation layer" combines the output of the "Flatten Layer" with the output of the respective AE of the MVL block to consider both the spatial and temporal data information during the learning of the model. This data information from the output of the "Concatenation Layer" is further learned by some consecutive "Dense Layers." This "Some Dense layers" is a hyperparameter that needs to be fixed based on the dataset and task. Moreover, the output layer of each task-specific head is basically a "Dense layer" with one node or neuron for the binary classification task with the activation function "Sigmoid" and n number of nodes for the multiclass classification with the activation function "Softmax." Here, n is the number of classes in the class label of the particular task. More details of the proposed model are provided in the Supplementary section.

1) *Algorithm for M²VT Approach:* The working of the M²VT approach is illustrated in Algorithm 1, and Table II shows the meanings of the symbols used in the algorithm. This approach requires a dataset D with features $F = \{f_1, f_2, \dots, f_n\}$ and tasks $T = \{t_1, t_2, \dots, t_n\}$, and

Fig. 1. M²VT: The proposed model.TABLE II
SYMBOLS USED

Symbols	Meaning
D	a dataset
F	set of features
f_j	jth feature in F
T	set of tasks
t_i	a task in T
P	set of predictions
p_{t_i}	prediction of task t_i
V	set of views
v_{t_i}	view of task t_i
SV	set of extracted spatial features, from $v_{t_i} \in V$, with reduced dimensions
$RF_{wrapper}$	a wrapper-based feature selection module with RF as estimator that follows the forward selection approach to find the relevancy of a f_j to a task t_i
sv_{t_i}	an extracted spatial feature from v_{t_i} for t_i , with reduced dimension
Tf	shared temporal features
AE	Autoencoder module for extracting spatial features and dimension reduction
$LSTM$	LSTM module for extracting the temporal features
C	a module to combine all the $v_{t_i} \in V$
$SV_{combine}$	combination of all $v_{t_i} \in V$
$Dense$	dense module for prediction

ensures performing multiple tasks. In step 1 of the algorithm, we have initialized the required parameters to keep the views (V), extracted spatial features from the views (V) with a lower dimension (SV), shared temporal features (Tf), and a combination of spatial views ($SV_{combine}$). In the next step, using the module $RF_{wrapper}$, we perform the wrapper-based feature selection using the RF classifier as an estimator to select the respective view v_i corresponding to the respective

task $t_i \in T$ from the original features set (F) of the dataset (D). To select a relevant feature set for a view v_i , the $RF_{wrapper}$ module applies the forward selection on the original feature set F in an incremental way. In each iteration of the incremental approach, the $RF_{wrapper}$ module considers the accuracy as the estimator of the corresponding features subset given to the $RF_{wrapper}$. Now, the $RF_{wrapper}$ module picks the best subset of features that yields the maximum accuracy as the highly relevant features of the view v_i corresponding to the task t_i . This way, step 2 creates the view $v_{t_i} \in V$ of each task $t_i \in T$. In step 3, we extract the spatial features sv_{t_i} from the view v_{t_i} for each task $t_i \in T$ using the AE module. The features extracted by the AE module yield lower dimensional feature space from the respective views. More specifically, the AE module takes each view v_{t_i} of the task $t_i \in T$ as input and extracts the spatial features of the task t_i as a latent space with a lower dimension from the respective input view v_{t_i} . Further, in step 4, the module C combines all the extracted spatial features $sv_{t_i} \in SV$ of the task $t_i \in T$ and stores the combined spatial features in $SV_{combine}$. Then, the $LSTM$ module extracts the temporal features by taking the combined spatial features $SV_{combine}$. This way, the shared temporal information among the tasks is extracted. Finally, in step 5, to perform the task $t_i \in T$, i.e., the prediction p_{t_i} of t_i , the $Dense$ module, utilizes the shared temporal features Tf with the respective extracted spatial features sv_{t_i} of t_i which are fed into the $Dense$ module as inputs. The $Dense$ module has a number of consecutive dense layers, where the number of layers and the corresponding output depend on the task t_i to be performed. In this way, the M²VT approach

Algorithm 1: M²VT Approach.

Require: A Dataset D with features $F = \{f_1, f_2, \dots, f_n\}$ and tasks $T = \{t_1, t_2, \dots, t_m\}$
Ensure: Predictions $P = \{p_{t_1}, p_{t_2}, \dots, p_{t_i}\}$ where p_{t_i} is the predictions of $t_i \in T$

```

1: Step 1: Initialization
2:  $V = \{\}$ 
3:  $SV = \{\}$ 
4:  $SV_{\text{combine}} = \{\}$ 
5:  $Tf = \{\}$ 
6: Step 2: Creating views
7: for  $t_i \in T$  do
8:    $v_{t_i} \leftarrow \{\}$ 
9:   for  $f_j \in F$  do
10:     $\text{relevant} \leftarrow RF_{\text{wrapper}}(f_j, v_{t_i}, t_i)$ 
11:    if  $f_j$  is relevant to  $t_i$  then
12:       $v_{t_i} \leftarrow f_j$ 
13:    end if
14:  end for
15:   $V \leftarrow v_{t_i}$ 
16: end for
17: Step 3: Spatial feature extraction and dimension reduction
18: for  $v_{t_i} \in V$  do
19:    $sv_{t_i} \leftarrow \text{AE}(v_{t_i})$ 
20:    $SV \leftarrow sv_{t_i}$ 
21: end for
22: Step 4: Temporal features extraction
23:  $SV_{\text{combine}} \leftarrow C(sv_{t_i} \in SV)$ 
24:  $Tf \leftarrow \text{LSTM}(SV_{\text{combine}})$ 
25: Step 5: Performing respective tasks (predictions)
26: for  $t_i \in T$  do
27:    $p_{t_i} \leftarrow \text{Dense}(Tf, sv_{t_i})$ 
28:    $P \leftarrow p_{t_i}$ 
29: end for

```

performs multiple tasks by utilizing multiple task-specific views.

2) *Complexity Analysis:* The complexity of the M²VT approach, i.e., Algorithm 1 is as follows: for initializing the parameters, such as V , SV , SV_{combine} , and Tf in step 1, the algorithm takes $O(1)$ time. In step 2 of the algorithm, in creating v_n numbers of views, the RF classifier estimator during wrapper feature selection on f_n number of features in F takes $O(f_n(f_n + 1)/2) \times O(RF_n)$. Here, $O(RF_n)$ time is consumed to check the relevance among the features using the RF estimator for n times. So, the total complexity of step 2 is $O(v_n) \times O(f_n(f_n + 1)/2) \times O(RF_n)$, i.e., $O(v_n \times (f_n^2 + f_n)/2 \times RF_n)$, which is equivalent to $O(f_n^2)$. Next, to extract the spatial features using AE, step 3 takes $O(i \times l \times h \times o)$, for the i number of inputs, l number of hidden layers, h number of neurons in each hidden layer of AE, and o number of outputs. So, the total complexity will be $O(v_n) \times O(i \times l \times h \times o)$, i.e., $O(v_n \times i \times l \times h \times o)$. This can be simplified as $O(x_1 \times y_1)$, where x_1 is $v_n \times i$ and y_1 is $l \times h \times o$. Similarly, to extract the shared temporal features using LSTM after combining the spatial features extracted by the individual AEs, the algorithm takes $O(i \times l \times h \times o)$ times. Therefore, the complexity of step 4 will be $O(1) + O(i \times l \times h \times o)$, i.e., $O(1 + i \times l \times h \times o)$. This can be simplified as $O(x_2 \times y_2)$, where x_2 is $i \times l$ and y_2 is $h \times o$. Finally, to predict the respective task as p_{t_i} corresponding to the number of $t_n \in T$ using the Dense module, the algorithm performs one task with a cost of $O(i \times l \times h \times o)$. Therefore, the complexity of performing t_n number of tasks in T will be $O(t_n) \times O(i \times l \times h \times o)$. This can be simplified as $O(x_3 \times y_3)$, where x_3 is $t \times i \times l$ and y_3 is $h \times o$.

So, the total complexity of our M²VT approach is $O(1) + O(f_n^2) + O(x_1 \times y_1) + O(x_2 \times y_2) + O(x_3 \times y_3)$. Taking the higher-order complexity, this can be written as $O(f_n^2)$.

IV. EVALUATION

The whole experiment is performed on a Windows 11 Pro workstation. This system has an Intel Xeon Gold 5220R processor (speed 2.2 GHz), 64GB RAM, and an NVIDIA T1000 4GB GPU card with 896 CUDA cores. We used Python (version 3.10.13) with Pandas, Numpy, Matplotlib, Scikit-learn, and Keras packages to build and analyse the proposed model. The experiment is carried out on four benchmark IoT datasets as well as on our testbed dataset.

A. Data Descriptions

In this article, we aim to build a smart attack detection system for securing IoT networks. So, we carried out experiments on a testbed IoT dataset collected by creating diverse attack scenarios with multiple IoT devices as well as on four benchmarks, publicly available IoT datasets, i.e., UNSW-BotIoT, X-IoTID, Edge-IoT, and N-BaIoT. The summary of these datasets is given in Table III. Moreover, our testbed architecture is illustrated in Fig. 2. The testbed comprises seven IoT devices, two Raspberry Pi units, an intelligent network switch, a wireless access point, multiple attacker systems, two normal user systems, a local server, and a network monitoring and traffic capturing system. The seven IoT devices are custom-built using ESP32 microcontrollers, integrating sensors, including temperature and humidity, photoresistor, ultrasonic distance measure, gas, water level, dust, and soil moisture sensors. The MQTT protocol is used for communication between the IoT devices. To simulate attacks, we employ Hping3² to generate DoS and DDoS attacks targeting IoT devices. Our monitoring system enables us to capture normal and attack traffic automatically in the presence and absence of attacks for three scenarios, namely: a) attack targeted to IoT applications; b) attack targeted to virtualization layers; and c) attack targeted to physical infrastructures. We collected the system's performance metrics, application metrics, and device status. We extract nine features with two class labels, containing 276 867 instances in total.

B. Model Training

The proposed model is trained using the selected feature subsets or views. Unlike the training of conventional DL models, training an MTL model is more tricky as there is more than one task with a respective loss function to be considered during the training. It is challenging to optimize the model weights to have minimum losses in all tasks simultaneously. To handle this, during the training of the proposed model, we used the transfer learning [32] concept for optimizing the weights to have a minimum loss. During the training, we first trained each AE separately using its respective feature subsets or views. We used "Keras Turner" and trained for 50 epochs with 32 "batch_size" to achieve the optimal dimension of the encoder output.

²<https://www.kali.org/tools/hping3/>

TABLE III
DATASET DESCRIPTION

Sl. No.	Datasets	Total No. of Instances		Total no. of Features		No. of Classes		
		Original	After Sampling	Original	After Feature Selection	Attack Detection	Attack Categories	Attack Types
1	UNSW-BoTIoT	3 668 522	729 368	43	18	2	5	8
2	X-IIoTID	820 834	1 004 188	65	45	2	10	19
3	Edge-IIoT	1 909 671	1 019 085	63	21	2	5	15
4	N-BaIoT	7 062 606	2 879 415	115	15	2		11
5	Testbed	276 867	559 095	9	8	2	3	

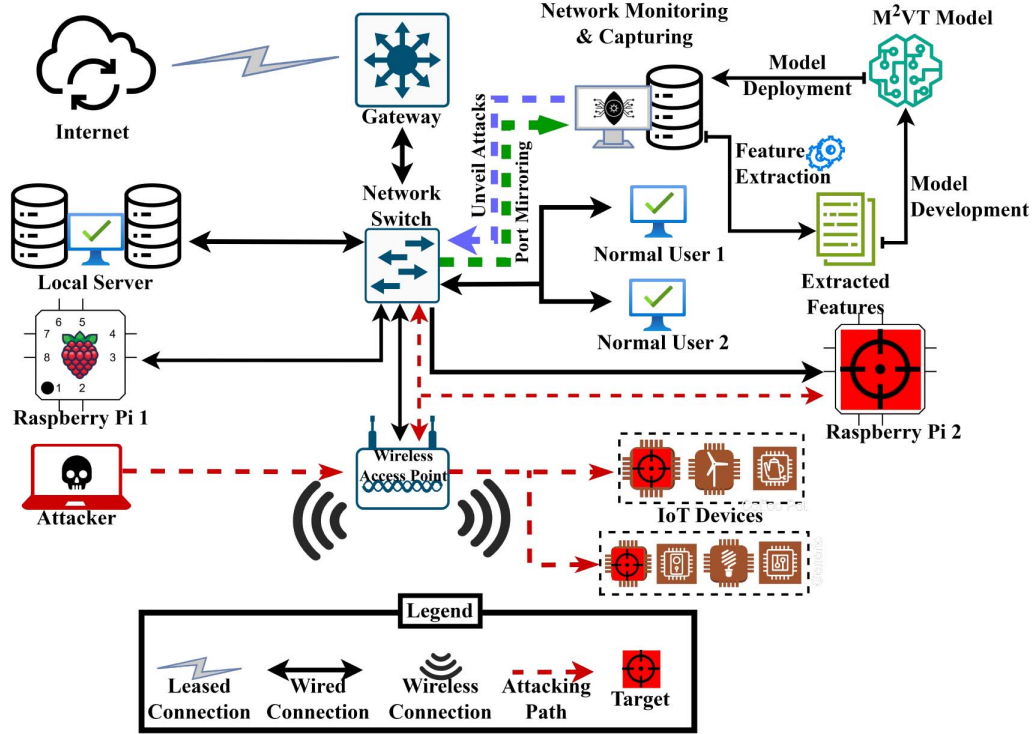


Fig. 2. Testbed setup.

After obtaining the favourable weights and encoder's output dimension of the AE, we change the layer parameters to be "nontrainable," which means the weight of the layer will not be updated in further training. Since training a DL model becomes challenging to achieve higher performance if the number of classes in the class label is higher, we trained the model for 2 or 3 (the number of tasks) consecutive times, starting by focusing on the task with the highest number of classes and gradually followed by focusing on the task with the lower number of classes to achieve high performance.

C. Result Analysis

The performance of M²VT is analyzed considering the commonly used evaluation measures, viz., accuracy (AC), precision (PR), recall (RC), F1-score (F1), Matthew's correlation coefficient (MCC), and receiver operating characteristic (ROC).

1) Comparison Against the Conventional DL Model: We compared the performance of the proposed M²VT model against the conventional DL model (single view–single task) based on the respective datasets. These conventional DL models are the segregated models from the proposed model. As presented in Table IV, the proposed model outperforms the conventional DL models in all respective tasks on all the datasets used in this experiment. However, during the experiment, we did not include the view and the task-specific head for the Edge-IIoT dataset to perform Task 1, i.e., attack detection, as we identified in the data preprocessing part, for attack detection, either the feature "mqtt.conack.flags-0.0," or "mqtt.protoname-0.0," or "mqtt.topic-0.0" can perfectly detect the attack (can classify the attacks from the normal) without any learning model. So, we keep the performance of both M²VT and the conventional DL model as 100% in all performance metrics for performing Task 1 of the Edge-IIoT dataset. Moreover, the ROC curve given by the M²VT in

TABLE IV
COMPARING M²VT WITH CONVENTIONAL DL MODEL

Sl. no.	Datasets	Task	M ² VT					Conventional DL				
			AC%	PR%	RC%	F1%	MCC%	AC%	PR%	RC%	F1%	MCC%
1	UNSW-BoTIoT	1	99.99	99.99	99.99	99.99	99.98	97.16	97.25	97.16	97.00	86.51
		2	98.49	98.48	98.49	98.48	97.80	45.9	56.58	45.9	42.65	40.58
		3	96.74	97.31	96.74	96.72	96.21	92.17	92.78	92.17	92.14	91.15
2	X-IIoTID	1	99.88	99.88	99.88	99.88	98.85	99.85	99.85	99.85	99.85	98.53
		2	99.91	99.91	99.91	99.91	99.90	98.78	98.78	98.78	98.78	98.60
		3	99.91	99.91	99.91	99.91	99.91	99.81	99.81	99.81	99.81	99.80
3	Edge-IIoT	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
		2	99.15	99.15	99.15	99.15	98.94	93.09	93.13	93.09	93.02	92.61
		3	99.19	99.20	99.19	99.19	99.13	93.56	93.61	93.56	93.57	91.87
4	N-BaIoT	1	99.97	99.97	99.97	99.97	99.81	99.95	99.95	99.95	99.95	99.71
		3	88.27	83.80	88.27	85.24	87.90	85.04	81.13	85.04	81.99	84.33
5	Testbed	1	99.32	99.32	99.32	99.32	98.46	96.62	96.62	96.62	96.62	92.35
		2	98.83	98.84	98.83	98.83	98.25	98.59	98.58	98.59	98.58	97.88

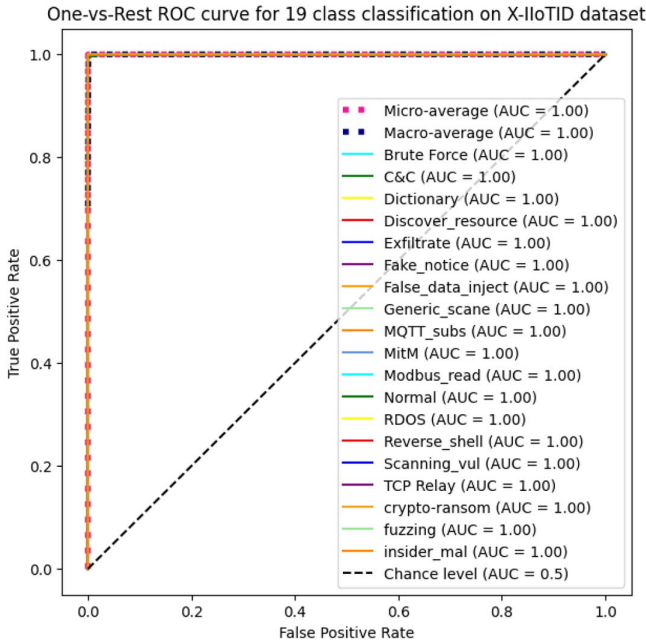


Fig. 3. ROC given by M²VT on performing task 3 of X-IIoTID dataset.

performing Task 3 of the X-IIoTID dataset is illustrated in Fig. 3. The figure represents each class's perfect classification in Task 3 of the X-IIoTID dataset. In addition, ROC curves given by the M²VT in the other used datasets are presented in the Supplementary section.

2) Comparison Against the State-of-the-Art Methods:

The performance of the proposed model is compared against the state-of-the-art methods based on the respective Tasks as follows.

a) *Performance comparison based on Task 1:* As shown in Table V, in performing Task 1 of the UNSW-BoTIoT dataset, the M²VT outperformed the compared methods

TABLE V
COMPARISON ON TASK 1

Dataset	Ref.	Method Name	AC%	PR%	RC%	F1%
UNSW-BoTIoT	[33]	LSVMS_RRBS	99.48	97.46	99.71	98.55
	[34]	RF-IFPCC	99.99	99.99	100.0	99.99
	[35]	ACOCN	99.96	100.0	99.96	99.98
	[35]	LPCOCN	99.97	100.0	99.97	99.98
	[36]	XGB	99.99	99.50	97.50	98.50
	[30]	M2VT-IDS	100.0	100.0	100.0	100.0
	[37]	CNN 3D	99.90	99.75	99.85	99.80
	[38]	SVM	99.99	99.99	100.0	86.58
	[39]	DNN	100.0	98.56	100.0	99.27
	[36]	RandomTree	99.99	100.0	100.0	100
	[22]	KNN-KBest	99.99	99.00	99.00	99.00
	[40]	RF	99.20	97.50	99.05	98.27
	[41]	FNN	95.00	95.00	95.00	95.00
	[42]	FNN	99.00	99.00	99.00	99.00
	Proposed	M ² VT	99.99	99.98	99.98	99.98
X-IIoTID	[43]	DT	99.54	99.54	99.54	99.54
	[43]	DNN	98.96	98.97	98.94	98.95
	[43]	GRU	99.46	99.46	99.45	99.45
	[44]	CNN	99.10	99.02	99.05	99.03
	[44]	LSTM	99.05	99.03	99.01	99.01
	[44]	CNN +LSTM	99.84	99.67	99.55	99.60
	[45]	RF	99.72	99.72	99.72	99.72
	[46]	PIGNUS	100.0	100.0	100.0	100.0
	[47]	AMPPSO+RF	98.81		98.85	
	Proposed	M ² VT	99.88	99.88	99.88	99.88
Edge-IIoT	[48]	J48	99.51	99.50	99.50	99.50
	[48]	PART	99.55	99.50	99.50	99.50
	[49]	RNN+LSTM	98.88	97.46	96.57	96.91
	[50]	DT	96.00	86.00	85.00	85.00
	[51]	DNN	99.99	99.99	99.97	99.98
	[28]	CNN+LSTM	100.0	100.0		
	Proposed	M ² VT	100.0	100.0	100.0	100.0
N-BaIoT	[52]	LSTM	99.98	100.0	99.98	99.98
	[39]	LSVMSRRBS	99.81	99.70	99.82	99.78
	[39]	DNN	99.99	100.0	99.98	99.98
	[53]	RF-SMOTE	99.98	100.0	99.97	99.98
	[54]	LR	99.98	99.90	99.96	99.92
	[54]	ANN	96.40	93.90	95.10	99.13
	[55]	NB-J48-ANN	99.10			
	[56]	CART	99.00			
	[57]	RNN	89.75			
	[58]	Collective DL	99.84			
	[59]	LGBA-NN	99.00			
	Proposed	M ² VT	99.97	99.97	99.97	99.97

and yields a similar performance with the RF-IFPCC [34], XGB [36], SVM [38], DNN [39], RandomTree [36], and KNN-KBest [22] methods. Moreover, the PR%,

TABLE VI
COMPARISON ON TASK 2

Dataset	Ref.	Method Name	AC%	PR%	RC%	F1%
UNSW-BoTIoT	[39]	DNN	99.96	99.2	99.96	99.57
	[60]	Hybrid IDS	99.97			
	[41]	FNN	95.00			
	[41]	SNN	91.00			
	Proposed	M ² VT	98.49	98.20	97.46	97.87
X-IIoTID	[43]	DT	99.49	97.33	97.20	97.27
	[43]	DNN	97.88	97.69	93.13	95.24
	[43]	GRU	99.47	97.11	96.69	96.89
	[46]	PIGNUS	97.90	88.00	95.00	88.23
	[61]	MLP - 3 - Layer	98.27	98.31	97.77	98.03
	[61]	MLP - 7 - Layer	98.10	98.07	97.56	97.80
	[62]	Residual - CNN	97.73	97.32	97.38	97.35
	[62]	Deep - CNN	96.65	96.67	97.29	96.93
	[63]	CNN - MLP	98.16	97.88	98.31	98.08
	[64]	L-CNN-MLP	98.41	97.35	98.09	97.70
	Proposed	M ² VT	99.91	99.91	99.91	99.91
Edge-IIoT	[65]	F-BIDS	90.91	92.83	93.83	93.00
	[51]	DNN	96.01	91.83	84.16	89.50
	[28]	CNN+LSTM	98.69	97.91		
	Proposed	M ² VT	99.15	99.23	99.28	99.25

TABLE VII
COMPARISON ON TASK 3

Dataset	Ref.	Method Name	AC%	PR%	RC%	F1%
UNSW-BoTIoT	[66]	DT	99.99			
	[66]	NB	97.50			
	[30]	M2VT-IDS	99.82	99.76	99.84	99.80
	Proposed	M ² VT	97.74	97.10	96.82	96.93
X-IIoTID	[43]	DT	99.45	94.16	93.54	93.80
	[43]	DNN	98.39	94.11	90.05	91.73
	[43]	GRU	99.46	96.47	91.58	93.54
	[44]	CNN	99.26	99.18	99.14	99.15
	[44]	LSTM	98.91	98.86	98.82	98.84
	[44]	CNN +LSTM	99.80	99.38	99.72	99.54
	[67]	CDAE-DNN	97.21			
	Proposed	M ² VT	99.91	99.91	99.91	99.91
Edge-IIoT	[68]	Inception Time	94.94	98.30	92.40	95.30
	[48]	J48	92.92	93.8	92.90	92.90
	[69]	MLP	99.98	99.95	99.98	99.96
	[65]	F-BIDS	90.93	88.06	92.20	89.00
	[51]	DNN	94.67	80.02	77.00	77.34
N-BaIoT	Proposed	M ² VT	99.19	99.20	99.19	99.19
	[70]	SNN	69.14	63.38	65.22	
	Proposed	M ² VT	88.27	83.80	88.27	85.24

RC%, and F1% values show the stability of M²VT in performing the task. Further, in performing Task 1 of the X-IIoTID dataset, the M²VT outperformed all the compared existing methods and performed similarly to PIGNUS [46]. Moreover, in the Edge-IIoT dataset, the M²VT outperformed all the compared existing methods and performed similarly to the CNN+LSTM [28] method. In addition, the M²VT gives similar performance to LSTM [52], DNN [39], RF-SMOTE [53], and LR [54] methods and outperformed the other compared methods in performing Task 1 of the N-BaIoT dataset.

- b) *Performance comparison based on Task 2*: As shown in Table VI, in performing Task 2 of the UNSW-BoTIoT dataset, the M²VT outperformed the FNN and SNN [41] methods. Moreover, M²VT outperformed all the compared existing methods in performing Task 2 of the X-IIoTID and Edge-IIoT datasets.
- c) *Performance comparison based on Task 3*: As shown in Table VII, in performing Task 3 of the UNSW-BoTIoT dataset, the M²VT outperformed the NB [66] method.

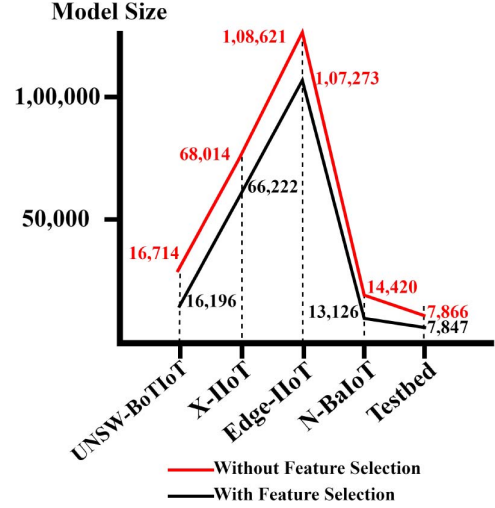


Fig. 4. Comparison on model size for each dataset, before and after feature selection.

TABLE VIII
COMPARISON BASED ON THE NUMBER OF MODEL SIZES

Ref.	Method Name	Dataset	Classification Type	No. of Parameters	AC%
[25]	PB-DID	UNSW-(NB15+BoTIoT)	Binary	59 530	99.00
			Multiclass (3)	59 551	96.30
[27]	CNN+LSTM	N-BaIoT	Multiclass (11)	610 315	88.83
[26]	TCNN	UNSW-BoTIoT	Multiclass (5)	42 629	99.99
[71]	DAE	N-BaIoT	Binary	36 279	
[39]	MLP	UNSW-BoTIoT	Binary	6945	100.00
			Multiclass (5)	40 037	99.96
			Multiclass (11)	46 859	100.00
			Binary	33 441	99.99
[9]	MS-DHRN	NSL-KDD	Multiclass(10)	19 306	99.99
			Binary	56 180	84.00
		USNW-NB15	Multiclass (5)	79.10	
			Binary	69 670	85.70
		CICIDS-2017	Multiclass (10)	76.00	
			Binary	58 510	91.40
Proposed	M ² VT	UNSW-BoTIoT	Multiclass (8)	90.40	
			Binary	99.99	
			Multiclass (5)	16 196	98.49
		X-IIoTID	Multiclass (8)	97.74	
			Binary	99.88	
			Multiclass (10)	66 222	99.91
		Edge-IIoT	Multiclass (19)	99.91	
			Multiclass (6)	107 273	99.15
			Multiclass (15)	99.19	
		N-BaIoT	Binary	13 126	99.97
			Multiclass (11)	88.27	
			Binary	7847	99.32
		Our Dataset	Multiclass (3)	98.83	

Moreover, the M²VT outperformed the compared existing methods in performing Task 3 of X-IIoTID, Edge-IIoT, and N-BaIoT datasets.

More detailed analyses, including performance comparisons based on ROC curves and class-wise prediction, are provided in the Supplementary section.

In this study, we did not include training and testing execution times to compare the efficiency of the proposed model against existing DL-based methods. These execution times are inherently influenced by system configurations and specifications, making them biased and inconsistent across different environments. Instead, we assessed the lightness of the proposed model based on its size, specifically the number of trainable parameters, as computational costs for DL-based methods are directly

TABLE IX
AVERAGE DETECTION TIME OF M²VT

Detection Time per Instance (in ms)														
Sl. No.	Datasets	GPU				CPU 2.2 GHz				CPU 3.5 GHz				
		Task 1	Task 2	Task 3	All Task at One Time	Task 1	Task 2	Task 3	All Task at One Time	Task 1	Task 2	Task 3	All Task at One Time	Task One
1	UNSW-BoTIoT	7	8	9	24	3	4	5	8	4	4	5	8	
2	X-IIoTID	13	14	15	40	7	6	8	15	5	5	7	14	
3	Edge-IIoT		14	16	33		7	8	14		6	7	12	
4	N-BaIoT	8		12	19	5	0	5	7	5		5	6	
5	Our Dataset	4	5		10	2	3		6	2	3		5	

tied to model size. While no existing DL-based methods in the literature are perfectly comparable to our proposed model, we compared it to some similar methods, as shown in Table VIII. The results demonstrate that the proposed model achieves a lower computational cost, performing 2 or 3 tasks simultaneously with a smaller model size, whereas the compared methods are limited to single-task performance. Additionally, Fig. 4 illustrates the impact of feature selection on model size. By using only the selected features, the proposed model is significantly smaller than when all features are used. Furthermore, as detailed in Table IX, we provide the average prediction time of the proposed model on both GPU and CPU systems. The experimental results indicate that the proposed model achieves faster detection times, particularly on a CPU with a clock speed of 3.5 GHz.

D. Discussion

Apart from the existing methods, which employ a common set of features to perform a single specific task, M²VT employs multiple relevant feature subsets to perform multiple tasks concurrently, which enhances the generalization of M²VT to yield higher performance. In this article, M²VT is tuned for three tasks but can be tuned for k number of tasks. Still, the model's performance will not drop because it will get more shared information during the learning. Although the proposed M²VT model has been evaluated in offline settings, its performance has been extensively tested using five diverse benchmark IoT datasets, each collected from distinct testbed settings and environments. The consistently high performance across these datasets demonstrates M²VT model's strong generalization capability, highlighting its potential scalability and adaptability to real-world IoT environments. Furthermore, M²VT is designed with computational efficiency and low model complexity, making it well-suited for deployment on resource-constrained IoT devices operating in real-time network environments. Although the training of the proposed model is more challenging than the training of conventional DL, we can see the effectiveness of the M²VT, outperforming the conventional DL models with a margin of 6.17% and the state-of-the-art methods with a margin of 1.47%. In addition, it is also possible to improve the performance of M²VT by tuning the model's hyperparameters. While searching for existing works to compare the proposed model's

performance, we found many methods for binary classification (*attack detection*). However, we can find only a limited number of works with little information about the details of their method, as well as their performance for the multiclass classification (classification of attack categories or classification of attack types). Although we have not yet deployed and evaluated the proposed model's performance in a real-time or near real-time IoT environment, the prediction time of M²VT depends on the architecture, which varies according to the employed data distribution and the system's configuration used for the model's deployment. However, from the experiment, M²VT can separately perform only Task 1 in an average of 4 milliseconds (ms), Task 2 in an average of 4.5 ms, Task 3 in an average of 6 ms, and concurrently all three tasks in an average of 7 ms. After performing rigorous experiments on the validation of M²VT, we found the following novel points of our method.

- 1) M²VT is a unique hybrid architecture that leverages MVL and MTL, incorporating AE and LSTM.
- 2) It concurrently considers multiple task-specific relevant features subsets to perform multiple tasks.
- 3) From the experimental analysis, M²VT performs better than the conventional DL methods. However, it gives higher and similar performance compared to the state-of-the-art methods.
- 4) M²VT has lower complexity than the existing state-of-the-art methods.

V. CONCLUDING REMARKS

Considering the wide applications given by IoT in today's world and its vulnerabilities, this article proposes a hybrid model called M²VT, leveraging MVL and MTL to detect and classify the attacks as well as their attack categories in an IoT network. The proposed model takes three relevant subsets of features as input views to perform three related tasks. The model used AE to extract spatial information from the input data and to reduce the dimension. Moreover, we also leverage LSTM to extract the shared temporal information among the tasks. The performance of the M²VT model is compared against the conventional DL method as well as with the state-of-the-art methods in terms of AC%, PR%, RC%, F1%, and MCC% on a testbed dataset as well as on four benchmark IoT datasets, viz., UNSW-BoTIoT, X-IIoTID, Edge-IIoT, and N-BaIoT. From the

experimental analysis, we can observe that M²VT outperformed the conventional DL models with a margin of 6.17% AC, 5.18% PR, 6.02% RC, 6.33% F1, and 7.38% MCC, as well as the state-of-the-art methods with a margin of 1.45% AC, 3.00% PR, 4.4% RC, and 3.59% F1. In the future, by deploying the M²VT model for detecting and classifying attacks, we will extend the work to prevent cyber-attacks in an IoT environment, which is not included in this work. Moreover, we will perform more experiments to improve the proposed model's performance and to evaluate the robustness of the model, such as by tuning the model's hyperparameters or incorporating other DL architectures like CNN and using more IoT benchmark datasets.

REFERENCES

- [1] N. Hoque, D. K. Bhattacharyya, and J. K. Kalita, "Botnet in DDoS attacks: Trends and challenges," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2242–2270, Apr. 2015.
- [2] N. Hoque, M. Bhuyan, R. Baishya, D. Bhattacharyya, and J. Kalita, "Network attacks: Taxonomy, tools and systems," *J. Netw. Comput. Appl.*, vol. 40, pp. 307–324, 2014.
- [3] M. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: Methods, systems and tools," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 303–336, Jan. 2014.
- [4] A. Behera, K. S. Sahoo, T. K. Mishra, and M. Bhuyan, "A combination learning framework to uncover cyber attacks in IoT networks," *Internet Things*, vol. 28, 2024, Art. no. 101395.
- [5] K. Bochie, M. S. Gilbert, L. Gantert, M. S. Barbosa, D. S. Medeiros, and M. E. M. Campista, "A survey on deep learning for challenged networks: Applications and trends," *J. Netw. Computer Appl.*, vol. 194, 2021, Art. no. 103213.
- [6] X. Yan, S. Hu, Y. Mao, Y. Ye, and H. Yu, "Deep multi-view learning methods: A review," *Neurocomputing*, vol. 448, pp. 106–129, 2021.
- [7] Y. Zhang and Q. Yang, "A survey on multi-task learning," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 12, pp. 5586–5609, Dec. 2022.
- [8] U. B. Clinton and N. Hoque, "MU-IoT: A new IoT intrusion dataset for network and application layer attacks analysis," *IEEE Access*, vol. 12, pp. 166068–166092, 2024.
- [9] H. He, X. Sun, H. He, G. Zhao, L. He, and J. Ren, "A novel multimodal-sequential approach based on multi-view features for network intrusion detection," *IEEE Access*, vol. 7, pp. 183207–183221, 2019.
- [10] D. C. Attota, V. Mothukuri, R. M. Parizi, and S. Pouriyeh, "An ensemble multi-view federated learning intrusion detection for IoT," *IEEE Access*, vol. 9, pp. 117734–117745, 2021.
- [11] S. Millar, N. McLaughlin, J. Martinez del Rincon, and P. Miller, "Multi-view deep learning for zero-day Android malware detection," *J. Inf. Secur. Appl.*, vol. 58, 2021, Art. no. 102718.
- [12] Y. Wang, W. Ma, H. Xu, Y. Liu, and P. Yin, "A lightweight multi-view learning approach for phishing attack detection using transformer with mixture of experts," *Appl. Sci.*, vol. 13, no. 13, 2023, Art. no. 7429.
- [13] M. Zhou, S. Lv, L. Yin, X. Chen, H. Li, and L. Sun, "SCTM: A multi-view detecting approach against industrial control systems attacks," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–6.
- [14] S. Ali, O. Abusabha, F. Ali, M. Imran, and T. Abuhmed, "Effective multitask deep learning for IoT malware detection and identification using behavioral traffic analysis," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1199–1209, Feb. 2023.
- [15] L. Nie, X. Wang, S. Wang, M. Obaidat, B. Sadoun, and S. Li, "Network traffic prediction in industrial Internet of Things backbone networks: A multi-task learning mechanism," *IEEE Trans. Ind. Informat.*, vol. 17, no. 10, pp. 7123–7132, Oct. 2021.
- [16] L. Sun, Y. Zhou, Y. Wang, C. Zhu, and W. Zhang, "The effective methods for intrusion detection with limited network attack data: Multi-task learning and oversampling," *IEEE Access*, vol. 8, pp. 185384–185398, 2020.
- [17] Q. Liu, D. Wang, Y. Jia, S. Luo, and C. Wang, "A multi-task based deep learning approach for intrusion detection," *Knowl.-Based Syst.*, vol. 238, 2022, Art. no. 107852.
- [18] W. Huang and J. W. Stokes, "MtNet: A multi-task neural network for dynamic malware classification," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, J. Caballero, U. Zurutuza, and R. J. Rodríguez, Eds. Springer Int. Publishing, 2016, pp. 399–418.
- [19] H. Huang, H. Deng, J. Chen, L. Han, and W. Wang, "Automatic multi-task learning system for abnormal network traffic detection," *Int. J. Emerg. Technol. Learn.*, vol. 13, no. 4, 2018.
- [20] B. Li, Y. Lin, and S. Zhang, "Multi-task learning for intrusion detection on web logs," *J. Syst. Archit.*, vol. 81, pp. 92–100, 2017.
- [21] M. Douiba, S. Benkirane, A. Guezzaz, and M. Azrour, "An improved anomaly detection model for IoT security using decision tree and gradient boosting," *J. Supercomputing*, vol. 79, no. 3, pp. 3392–3411, 2023.
- [22] M. Mohy-Eddine, A. Guezzaz, S. Benkirane, and M. Azrour, "An efficient network intrusion detection model for IoT security using K-NN classifier and feature selection," *Multimedia Tools Appl.*, vol. 45, pp. 1–19, 2023.
- [23] Q. Abu Al-Haija and M. Al-Dala'ien, "ELBA-IoT: An ensemble learning model for botnet attack detection in IoT networks," *J. Sensor Actuator Networks*, vol. 11, no. 1, 2022.
- [24] C. Hazman, A. Guezzaz, S. Benkirane, and M. Azrour, "IIDS-SIoEL: Intrusion detection framework for IoT-based smart environments security using ensemble learning," *Cluster Comput.*, vol. 26, no. 6, pp. 4069–4083, 2023.
- [25] M. Zeeshan et al., "Protocol-based deep intrusion detection for DoS and DDoS attacks using UNSW-NB15 and Bot-IoT data-sets," *IEEE Access*, vol. 10, pp. 2269–2283, 2022.
- [26] A. Derhab, A. Aldweesh, A. Z. Emam, and F. A. Khan, "Intrusion detection system for Internet of Things based on temporal convolution neural network and efficient feature engineering," *Wireless Commun. Mobile Comput.*, vol. 2020, pp. 1–16, 2020.
- [27] H. Alkhatani and T. H. Aldhyani, "Botnet attack detection by using CNN-LSTM model for Internet of Things applications," *Secur. Commun. Networks*, vol. 2021, pp. 1–23, 2021.
- [28] A. Khacha, R. Saadouni, Y. Harbi, and Z. Aliouat, "Hybrid deep learning-based intrusion detection system for industrial Internet of Things," in *Proc. 5th Int. Symp. Inform. Its Appl. (ISIA)*, 2022, pp. 1–6.
- [29] U. B. Clinton, N. Hoque, and K. R. Singh, "Classification of DDoS attack traffic on SDN network environment using deep learning," *Cybersecurity*, vol. 7, no. 1, 2024, Art. no. 23.
- [30] F. Nie, W. Liu, G. Liu, and B. Gao, "M2VT-IDS: A multi-task multi-view learning architecture for designing IoT intrusion detection system," *Internet Things*, vol. 25, 2024, Art. no. 101102.
- [31] R. Caruana, "Multitask learning," *Mach. Learn.*, vol. 28, pp. 41–75, 1997.
- [32] L. Torrey and J. Shavlik, "Transfer learning," in *Handbook of Research on Machine Learning Applications and Trends: algorithms, Methods, and Techniques*. IGI global, 2010, pp. 242–264.
- [33] J. Azimjonov and T. Kim, "Designing accurate lightweight intrusion detection systems for IoT networks using fine-tuned linear SVM and feature selectors," *Comput. Secur.*, vol. 137, 2024, Art. no. 103598.
- [34] M. Mohy-Eddine, A. Guezzaz, S. Benkirane, M. Azrour, and Y. Farhaoui, "An ensemble learning based intrusion detection model for industrial IoT security," *Big Data Mining Anal.*, vol. 6, no. 3, pp. 273–287, 2023.
- [35] B. A. N. Gopinathan, V. Sarveshwaran, V. Ravi, and R. Chaganti, "LPCOCN: A layered paddy crop optimization-based capsule network approach for anomaly detection at IoT edge," *Inf. (Basel)*, vol. 13, no. 12, 2022, Art. no. 15.
- [36] M. Shafiq, Z. Tian, Y. Sun, X. Du, and M. Guizani, "Selection of effective machine learning algorithm and Bot-IoT attacks traffic identification for Internet of Things in smart city," *Future Gener. Computer Syst.*, vol. 107, pp. 433–442, 2020.
- [37] I. Ullah and Q. H. Mahmoud, "Design and development of a deep learning-based model for anomaly detection in IoT networks," *IEEE Access*, vol. 9, pp. 103906–103926, 2021.
- [38] A. Guezzaz, A. Asimi, Y. Sadqi, Y. Asimi, and Z. Tbatou, "A new hybrid network Sniffer model based on Pcap language and sockets (Pcapsocks)," *Int. J. Adv. Comput. Sci. Appl.*, vol. 7, no. 2, pp. 131–145, 2016.
- [39] S. Popoola, B. Adebisi, G. Gui, M. Hammoudeh, H. Gacanin, and D. Dancy, "Optimizing deep learning model hyperparameters for botnet attack detection in IoT networks," 2022, *arXiv:1950.1885.v1*.
- [40] R. Pavaiyarkarasi, T. Manimegalai, S. Satheshkumar, K. Dhivya, and G. Ramkumar, "A productive feature selection criterion for Bot-IoT

- recognition based on random forest algorithm," in *Proc. 11th Int. Conf. Commun. Syst. Netw. Technol.*, 2022, pp. 539–545.
- [41] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," in *Proc. IEEE Global Commun. Conf.*, 2019, pp. 1–6.
 - [42] M. Ge, X. Fu, N. Syed, Z. Baig, G. Teo, and A. Robles-Kelly, "Deep learning-based intrusion detection for IoT networks," in *Proc. IEEE 24th Pacific Rim Int. Symp. Dependable Comput. (PRDC)*, 2019, pp. 256–25609.
 - [43] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "X-IIoTID: A connectivity-agnostic and device-agnostic intrusion data set for industrial Internet of Things," *IEEE Internet Things*, vol. 9, no. 5, pp. 3962–3977, May 2022.
 - [44] H. C. Altunay and Z. Albayrak, "A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks," *Eng. Sci. Technol. Int. J.*, vol. 38, 2023, Art. no. 101322.
 - [45] H. Shah et al., "Deep learning-based malicious smart contract and intrusion detection system for IoT environment," *Mathematics*, vol. 11, no. 2, 2023.
 - [46] P. Jayalaxmi, R. Saha, G. Kumar, M. Alazab, M. Conti, and X. Cheng, "PIGNUS: A deep learning model for IDS in industrial internet-of-things," *Comput. Secur.*, vol. 132, 2023, Art. no. 103315.
 - [47] H. Zhao et al., "An enhanced intrusion detection method for AIM of smart grid," *J. Ambient Intell. Humanized Comput.*, vol. 14, no. 5, pp. 1–13, 2023.
 - [48] T. Al Nuaimi et al., "A comparative evaluation of intrusion detection systems on the edge-IIoT-2022 dataset," *Intell. Syst. Appl.*, vol. 20, 2023, Art. no. 200298.
 - [49] A. A. Alashhab, M. S. M. Zahid, A. Muneer, and M. Abdullahi, "Low-rate DDoS attack detection using deep learning for SDN-enabled IoT networks," *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 11, 2022.
 - [50] G. Bhandari, A. Lyth, A. Shalaginov, and T.-M. Grønli, "Distributed deep neural-network-based middleware for cyber-attacks detection in smart IoT ecosystem: A novel framework and performance evaluation approach," *Electronics*, vol. 12, no. 2, 2023.
 - [51] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
 - [52] U. B. Clinton, N. Hoque, and K. R. Singh, "Botnet-based IoT network attacks identification using LSTM," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol.*, 2023, pp. 1–10.
 - [53] M. G. Karthik and M. M. Krishnan, "Hybrid random forest and synthetic minority over sampling technique for detecting Internet of Things attacks," *J. Ambient Intell. Humanized Comput.*, vol. 23, no. 5, pp. 1–11, 2021.
 - [54] F. Abbasi, M. Naderan, and S. E. Alavi, "Anomaly detection in Internet of Things using feature selection and classification based on logistic regression and artificial neural network on N-BaIoT dataset," in *Proc. 5th Int. Conf. Internet Things Appl.*, 2021, pp. 1–7.
 - [55] Y. N. Soe, Y. Feng, P. I. Santosa, R. Hartanto, and K. Sakurai, "Machine learning-based IoT-botnet attack detection with sequential architecture," *Sensors*, vol. 20, no. 16, 2020.
 - [56] C. S. Htwe, Y. M. Thant, and M. M. S. Thwin, "Botnets attack detection using machine learning approach for IoT environment," *J. Phys. Conf. Ser.*, vol. 1646, Sep. 2020, Art. no. 012101.
 - [57] A. A. Hezam, S. A. Mostafa, A. A. Ramli, H. Mahdin, and B. A. Khalaf, "Deep learning approach for detecting botnet attacks in IoT environment of multiple and heterogeneous sensors," in *Proc. Int. Conf. Advances in Cyber Secur.* Springer, 2021, pp. 317–328.
 - [58] T. V. Khoa et al., "Collaborative learning model for cyberattack detection systems in IoT industry 4.0," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2020, pp. 1–6.
 - [59] A. Alharbi, W. Alosaimi, H. Alyami, H. T. Rauf, and R. Damaševičius, "Botnet attack detection using local global best bat algorithm for industrial Internet of Things," *Electronics*, vol. 10, no. 11, 2021.
 - [60] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman, and A. Alazab, "A novel ensemble of hybrid intrusion detection system for detecting Internet of Things attacks," *Electronics*, vol. 8, no. 11, pp. 134–156, 2019.
 - [61] T. Dong, S. Li, H. Qiu, and J. Lu, "An interpretable federated learning-based network intrusion detection framework," 2022, *arXiv:2201.03134*.
 - [62] Q. Tian, C. Guang, C. Wenchao, and W. Si, "A lightweight residual networks framework for DDoS attack classification based on federated learning," in *Proc. INFOCOM or IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
 - [63] W. Liu et al., "Intrusion detection for maritime transportation systems with batch federated aggregation," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, pp. 2503–2514, Feb. 2023.
 - [64] A. Zainudin, R. N. Alief, M. A. P. Putra, R. Akter, D.-S. Kim, and J.-M. Lee, "Blockchain-based decentralized trust aggregation for federated cyber-attacks classification in SDN-enabled maritime transportation systems," in *Proc. IEEE Int. Conf. Commun. Workshops*, 2023, pp. 182–187.
 - [65] O. Aouedi and K. Piamrat, "F-BIDS: Federated-blending based intrusion detection system," *Pervasive Mob. Comput.*, vol. 89, 2023, Art. no. 101750.
 - [66] M. Shafiq, Z. Tian, A. K. Bashir, X. Du, and M. Guizani, "CorrAUC: A malicious Bot-IoT traffic detection method in IoT network using machine-learning techniques," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3242–3254, May 2021.
 - [67] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "Asynchronous peer-to-peer federated capability-based targeted ransomware detection model for industrial IoT," *IEEE Access*, vol. 9, pp. 148738–148755, 2021.
 - [68] I. Tareq, B. M. Elbagoury, S. El-Regaily, and E.-S. M. El-Horbaty, "Analysis of ToN-IoT, UNW-NB15, and edge-IIoT datasets using DL in cybersecurity for IoT," *Appl. Sci.*, vol. 12, no. 19, 2022.
 - [69] A. Abid, F. Jemili, and O. Korbaa, "Real-time data fusion for intrusion detection in industrial control systems based on cloud computing and big data techniques," *Cluster Comput.*, vol. 56, no. 5, pp. 1–22, 2023.
 - [70] M. Umair, W.-H. Tan, and Y.-L. Foo, "Efficient malware classification with spiking neural networks: A case study on N-BaIoT dataset," in *Proc. 14th Int. Conf. Ubiquitous Future Networks (ICUFN)*, 2023, pp. 231–236.
 - [71] Y. Meidan et al., "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Mar. 2018.
 - [72] R. Alanazi and A. Aljuhani, "Anomaly detection for industrial Internet of Things cyberattacks," *Comput. Syst. Sci. Eng.*, vol. 44, no. 3, 2023.
 - [73] S. I. Popoola, B. Adebisi, M. Hammoudeh, G. Gui, and H. Gacanin, "Hybrid deep learning for botnet attack detection in the internet-of-things networks," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4944–4956, Jun. 2021.
 - [74] B. Majhi, "An improved intrusion detection system using BoT-IoT dataset," in *Proc. 11th Int. Conf. Commun. Syst. Netw. Technol.*, 2022, pp. 488–492.
 - [75] S. Selvarajan et al., "An artificial intelligence lightweight blockchain security model for security and privacy in IIoT systems," *J. Cloud Comput.*, vol. 12, no. 1, 2023, Art. no. 38.



Urikhimbam Bobby Clinton (Member, IEEE) is currently working toward the Ph.D. degree in computer science with the Department of Computer Science, Manipur University, Imphal, India.

He was also a Junior Research Fellow on a fully government-funded project. He is also a Visiting Faculty member with the Department of Computer Science, Manipur University. His research interests include the Internet of Things, computer networking, cybersecurity, software-defined networking, and artificial intelligence.



Nazrul Hoque received the Ph.D. degree in computer science and engineering from Tezpur University, Tezpur, India, in 2017.

He is currently working as an Assistant Professor with the Department of Computer Science, Manipur University, Imphal, India. His research interests include machine learning, network security, and the Internet of Things.

Dr. Hoque has published more than 40 papers in international journals and referred to conference proceedings.



Shahid Raza (Senior Member, IEEE) received the M.Sc. degree in cybersecurity from KTH Royal Institute of Technology, and the Ph.D. degree in Internet of Things (IoT) security from Mälardalen University.

He is a cybersecurity Researcher specializing in IoT security and serves as the Chair and a Full Professor of cybersecurity with the University of Glasgow, Scotland, U.K. He is also affiliated with RISE, Kista, Sweden; and Cybercampus, Stockholm, Sweden, where he serves as a Research

Director of cybersecurity, and is a Professor with Mälardalen University, Västerås, Sweden. He is an Expert in cybersecurity, primarily focusing on the security of the IoT.

Dr. Raza's scientific work was published in leading journals and conferences. Previously, he was an Associate Professor (Swedish Docent) with Uppsala University, Uppsala, Sweden.



Monowar Bhuyan (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from Tezpur University, Assam, India, in 2014.

He is an Associate Professor with the Department of Computing Science, Umeå University, Umeå, Sweden. He established and leads the Cyber Analytics and Learning Group, Umeå, Sweden, and is a Senior Member with the Autonomous Distributed Systems Laboratory, Umeå. Prior to this, he worked at various academic institutions, including Nara In-

stitute of Science and Technology, Nara, Japan; Assam Kaziranga University, Jorhat, Assam, India; and Umeå University, at different levels, from a Junior Scientist to an Associate Professor, from 2009 to 2019. His research interests include machine learning, anomaly detection, systems and AI security, and distributed systems.

Dr. Bhuyan has an extensive publication record, with over 100 papers in the leading international journals and conference proceedings and he is the author of the book titled *Network Traffic Anomaly Detection and Prevention* (Springer). His experience leading/coleading research projects attracted over 40 MSEK from national, European Commission, and international grant agencies. He is a WASP Fellow.